

嵌入式小记----xby

一、概述

嵌入式系统的概念

- 定义：
 - 一个**大系统**中集成的一部分
 - 通过特定的硬件装置来**控制**该系统
- 组成
 - **嵌入式系统硬件**：整个系统的载体，以嵌入式处理器为核心
 - **嵌入式操作系统**：管理嵌入式硬件资源并进行服务能力扩展的系统级软件，即提供可定制能力和性能保证
 - **嵌入式应用软件**：面向特定应用所开发的软件系统
- 特点
 - 专用性强
 - 资源定制化
 - 非功能属性约束多
 - 资源相对受限
 - 一体化设计
 - 技术途径丰富
 - 知识与技术密集

二、嵌入式系统硬件

- 浮动信号：正常来说，电路信号高或低。如果引脚不受任何元件控制，则电压随时间不确定地变化
- 三态门：输出可以是高电压、低电压、浮动（高阻状态、断路）
- 上拉电阻、下拉电阻
- 二极管：两个极、一个PN结、单向导电、正向导通
- 三极管：双极型晶体管、基于输入电流控制输出电流、物理上实现数字逻辑的关键、包括NPN和PNP

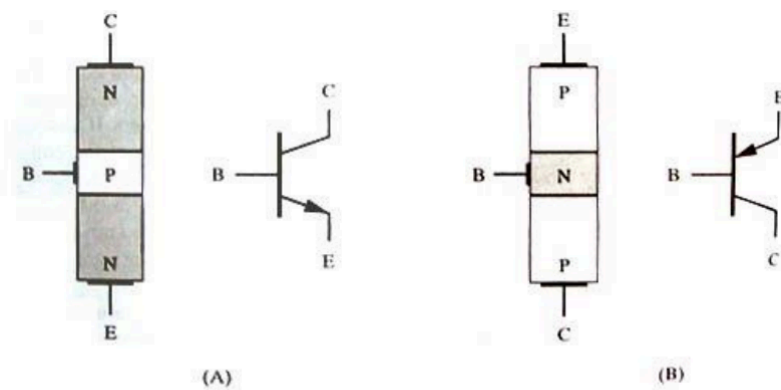


图 1 NPN 晶体管 (A) 和 PNP 晶体管 (B) 的电路图符号和结构图

- 场效应管 (FET) : JFET和MOS-FET
- 集电极开路 (OC门、NPN型三极管、开放收集器)
- ppt 27页的例题分析
- 漏极开路 (OD门) : 将OC门中的三极管换成N-MOS管, 并以其漏极作为输出
- 推挽电路: 通过自身内部的两个晶体管来输出高或低。不可以线与 (比如A输出高 (上通, 下截止。连接VCC), B输出低 (上截止, 下通。连接GND), 这使得VCC到GND有大电流), 不可以转换电平 (由芯片内部决定的电压)

OD门输入高---MOS管导通---输出低

OD门输入低---MOS管截止---输出高阻态浮空---上拉电阻实现高电平

OD门支持 线与 功能 (可以改变外部VCC来改变电平)

OC被OD淘汰, 因为前者是电流控制元器件, 后者是电压控制元器件, 前者容易被烧坏

- 退耦电容: 用于退耦 (消除电路之间的寄生耦合: 也就是电路A变化的时候, 会对电路B造成影响, 会有反冲电流), 基于滤波作用来消除电路回波噪声, 或者基于储能来快速提供能量补充

在信号输出引脚, 将输出信号的干扰作为滤除对象

在电路板上离可能需要大量电量的电路较近的地方

靠近电源的管脚, 电容的电源走线和地线所包围的面积最小

- 旁路电容: 把输入信号中的高频噪声作为滤除对象

三、嵌入式处理器子系统

- 微控制器MCU：即单片机
- 嵌入式微处理器MPU
- DSP
- CPLD FPGA
- SOC
- 众核计算单元

阿姆达尔定律：计算系统的加速比与任务的并行度以及处理器的数量成正比

四、嵌入式存储器子系统

读操作的基本过程：

- 处理器对存储器的访问：先地址有效，再数据访问的基本逻辑次序
- 两次读取操作之间的最小时间间隔就是 读取周期 $t_{RC} = \text{读取时间}t_A + \text{读恢复时间}t_{OTD}$

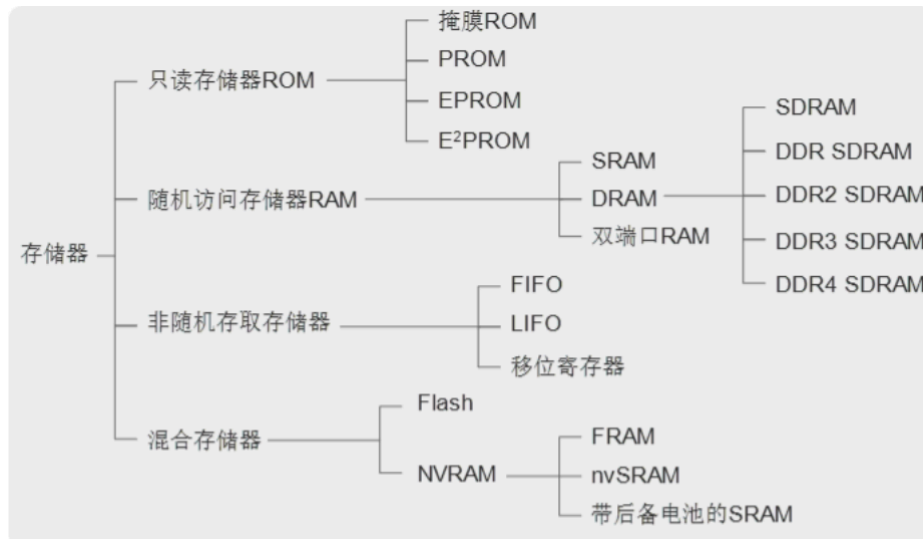
写操作的基本过程：

- 先地址有效，再数据访问
- 写操作之间的最小时间间隔：写周期 $t_{WC} = t_{AW} + t_W + t_{WR}$

存储器的性能指标：

- 只读性
 - 只读存储器ROM（非易失性）
 - 随机存储器RAM（易失性）
- 易失性
- 位容量
 - $4K * 1$ ：读写4K内存单元的某一位，芯片有一个数据输入端和一个数据输出端
 - $1K * 4$ ：读写1K内存单元的某四位，芯片有四个数据输入端和四个数据输出端
 - 二者的位容量是相同的
- 速度：用 存储器访问时间（存储器从接收到稳定的地址输入到完成操作的时间）来衡量
- 功耗
 - 双极型：速度快，功耗高
 - CMOS：速度慢，功耗低
- 可靠性
- 价格

嵌入式系统基本都使用----半导体存储器



- 随机存储器：可被 随机读或写，与磁盘不同，允许以任意次序读或写
 - SRAM：芯片有电---内容存在，掉电---内容丢失。快，耗电多
 - DRAM：数据寿命短，即使连续供电，需要周期性刷新。存储密度大

SRAM是基于双极性晶体管/MOS晶体管的静态RS逻辑触发器

128 K *8 的SRAM： 17条地址线， 8条数据线

DRAM： 存储元以电容中的电荷存储状态来表示0 1

电容中的电量会发生泄漏----即使连续供电----DRAM中的信息也会因电荷泄露而丢失

为什么DRAM效率低：

- CPU访问DRAM和DRAM刷新可能冲突----刷新期间DRAM控制器向CPU发出DRAM忙信号----CPU插入等待周期----降低系统性能

技术原理：

- 页模式（异步）
- EDO（异步）： 扩展的数据输出
- 同步 DRAM： DRAM中引入时钟， SDRAM

双端口RAM（DPRAM）： 一套存储体 两套独立访问端口

只读存储器

- 掩模ROM： 内容在芯片生产出之前就定，大批量生产的成本低，分为MOS型（慢）和双极型（快）
- PROM： 一次性编程，一旦其被编程了，内容就再也不能改变 广泛使用OTP

- EPROM：可多次进行擦除和重写，紫外线擦除，写入过程慢
- EEPROM：和EPROM类似，但更贵，擦除完全靠电力，写入周期比RAM长

混合存储器：RAM的快速读写，不挥发性，介于RAM和ROM之间

- 通电和SRAM一样
- 断点后保存其内容
- 速度快
- 价格高

FLASH：单电压的EEPROM，在其电路板上直接擦除和重新编程，一次能擦除一个扇区，而不是逐字节擦除

- 高密度 低价格 非易失性 快速（读取 而不是写入） 电可重编程
- 与传统存储器的优点对比：固有不挥发性，易更新性，可重复写次数多
- 原理：用存储元的源、漏极是否导通来表示 0 1
- 分类：NOR NAND
 - 内部存储体的结构
 - 接口不同：
 - NOR----SRAM
 - NAND----I/O
 - 随机读取速度不同：
 - NOR可以取代EEPROM，多用于BOOT ROM
 - NAND----高密度，多用于大量data存储

NOR FLASH：在EEPROM基础发展的，存储单元由NMOS构成。随机读取任意单元内容，常用于制作BIOS存储器和微控制器内部存储器

NAND FLASH：将几个N-MOS单元用同一根线连接，顺序读取存储单元内容，适合数据或文件的串行读写存储

NAND FLASH的特点：

- 更高存储密度、更快写入速度、更低的价格、更好的擦写耐用性，适用于大量数据的存储
- 接口和操作复杂，位交换操作频繁，通常采用错误探测/错误纠正（EDC/ECC）
 - 比如三星某FLASH：每页512字节且预留16字节的错误校验码存放空间
- 地址信息包括：列地址、块地址、相应的页面地址
 - 这些地址需要8位总线分组传输，需要多个时钟周期，当容量增大的时候，地址信息增加，那么就需要占用更多的寻址周期，寻址时间也就越长，因此不适合于频繁、小数据量访问的应用

FLASH操作

- 读操作：处理器只要提供 地址 读操作 片选，存储器返回数据
- 写操作：
 - 每一个存储位置必须在重写之前被擦除，否则结果将可能是新旧值的某个逻辑组合
 - 一次擦除一个扇区块，不可能只擦除单个字节，扇区大小随具体器件变化，通常是KB量级
 - 擦除和写入数据的过程取决于器件特性

- FRAM铁电存储器
- BBSRAM
- nvSRAM

五、最小系统及外围电路

最小系统：一个仅具有进入正确执行模式所需最少资源的系统

供电电路

- 嵌入式系统硬件的基本组成，为系统提供一种或多种负载能力的电压输出
- 稳定性重要
- 采用 直流稳压电源电路

复位电路

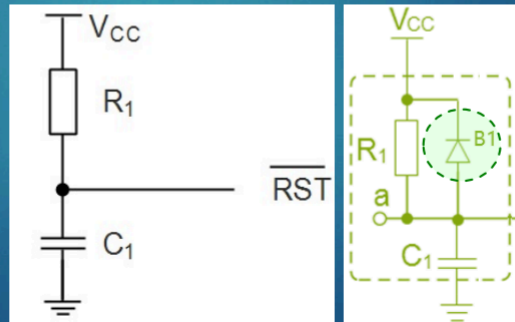
- 用途：在系统电源建立过程中，为CPU或某些接口电路提供一个复位脉冲，系统振荡器启动并稳定下来，CPU复位内部寄存器及指针
- 形式：
 - 阻容复位电路
 - 手动复位
 - 看门狗复位
 - 专用复位电路
 - 内部复位
 - 软件复位
- 上电复位电路（POR）：
 - 电源电压达到可以正常工作的阈值电压时，集成电路内部的状态便可以初始化器件，使整个芯片在上电后的一段时间内进入已知状态
 - 在完成初始化之前，忽略除了复位引脚之外的任何外部信号
- 阻容复位电容：利用电容电压不能突变 而是按指数规律上升/下降----配合反相器的输入电平翻转门限产生所需的复位脉冲

- 复位脉冲宽度取决于电容 电阻的参数。
- 缺点：

► 缺点

- 在电源瞬时跌落和恢复过程中，电容充放电的指数特性导致电容未完成放电时便又开始充电，从而无法产生合格宽度的复位脉冲；

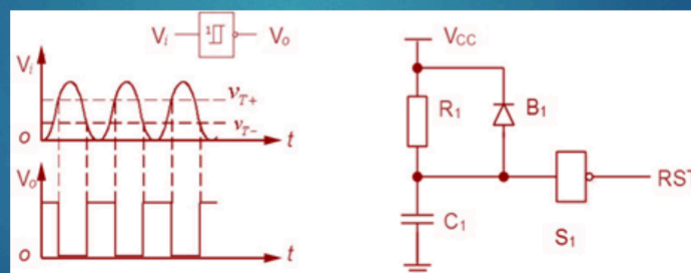
► 如何改进？



可以加一个二极管，当电源正常的时候，二极管反向截止，不影响RC正常充电去产生复位信号。当电源瞬间跌落时候，二极管正向导通，为电容提供一个快速放电通道。电源恢复的时候，电容能更快地回到初始状态，重新开始正常充电过程，从而产生合格宽度的复位脉冲。

- 进一步：为了提高复位电路的抗干扰能力，也可在阻容式复位电路的输出端接一个具有抗干扰能力的施密特器件S1。

- 利用施密特器件的迟滞特性，可以在一定程度上消除微小输入电压变化所可能引起的输出电压改变。



(a) 基于施密特触发器的反相器 (b) 接施密特反相器的复位电路
图5.14 施密特反相器及改进的上电复位电路

施密特器件的迟滞特性：他有两个不同的阈值电压，正向阈值电压 V_{T+} ，负向阈值电压 V_{T-} 。

当 V 超过 V_{T+} ：高变低。当 V 低于 V_{T-} 的时候：低变高。

这可以抗干扰，只有当输入电压变化超过一定范围（从低于 V_{T-} 到高于 V_{T+} ，或者从高于 V_{T+} 到低于 V_{T-} ）时，输出才会改变，

- 手动复位：

- 通常配合自动复位电路工作
- 手动复位开关产生的复位信号接在复位电路上，而不直接接在处理器复位信号输入端上，可以消除开关抖动（因为电容电压不能突变，开关抖动产生的瞬间电压变化，相对于电容缓慢的充电过程影响很小）
- 专用复位电路
 - 一种专用的集成电路，输出复位信号
 - 可支持多种复位信号
 - 所产生的复位信号质量较高
- 看门狗复位电路
 - 设计一个状态监控电路来监测 控制 运行状态----让系统在故障的时候自动 重新启动
 - 在故障的时候 向RST段输出复位信号
 - 工作原理：
 - 特定硬件，软硬件配合
 - 嵌入式系统程序 正常逻辑 每个确定的时间必然运行其中的某一部分代码，完成看门狗复位操作
 - 如果在一个特定最大时间内没有该类代码，则认为系统故障，看门狗电路产生复位信号，使处理器强制复位，系统从复位向量处重新运行
- 内部复位
 - 利用看门狗复位方式
 - 上电时候 程序没有复位看门狗 看门狗计时器溢出 完成复位
 - 微控制器不需要reset引脚
- 软件复位
 - 通过软件设置一个特殊功能寄存器的位完成控制器的复位，复位结果和硬件复位一样，部分MCU有
 - 注意
 - 软件复位与程序从复位向量处开始运行不同
 - 从复位向量处开始运行程序，处理器或控制器的状态不会回到复位状态，只是从start处开始运行程序

时钟电路

动态电压调节，动态时钟频率以及控制启用的外设数量 等是优化系统性能 降低功耗的重要方式

- 调节电压 控制频率
- 选择时钟源

电路抖动与消抖

消除抖动：

- 优化硬件设计
 - RC消抖电路
 - 基于RS双稳态触发器的电路消抖

- 专用消抖器件
- 基于采样统计思想的软件消抖
- 噪声识别消抖
 - 耦合噪声信号的特点之一：宽度极小于正常信号
 - 上升沿计时 下降沿计时 计算信号时长

硬件消抖是根本方式

缺点：

- 增加硬件设计的复杂度和成本
- 当硬件中噪声不可避免时，优化硬件可能无法解决问题，同时由于硬件的加工，调试会使整个项目周期显著延长

例题：

- 消除电路抖动的方法：RC消抖电路 专用消抖器件 延迟采样
- 看门狗复位电路：
 - 实现嵌入式系统的自动状态监测和恢复
 - 可以针对嵌入式处理器运行过程中可能出现死机和跑飞的情况
 - 需要软硬件配合
 - 一旦停止喂狗，立即复位 **(这是错误的)**
 - reason：喂狗是指软件定期向看门狗定时器发信号，重置定时器计时。停止喂狗的时候，当看门狗定时器计时溢出（超过设定的定时时间）才会复位
- 低功耗设计：
 - 动态电压调节
 - 动态时钟频率选择
 - 对代码编译优化
 - 控制启用的外设数量

六、接口与总线

- 专用I/O限制了处理器的功能和使用范围
- 灵活性差，且接口多时芯片引脚增加

增强I/O接口的通用性-----GPIO-----可适用多种输入输出要求-----用户可以根据不同功能需求，通过寄存器组，进行I/O的动态配置和管理

GPIO操作流程：

- 使能相关GPIO口时钟

- 配置GPIO管脚模式
 - 设置管脚方向
 - 读管脚（输入）
-

I2C：集成电路总线

- 主从式
- 可寻址
- 2线制
- 近距离
- 串行通信总线

两条双向线：

- SDA：双向数据线 OD门
- SCL：上升沿 下降沿 传输数据（边沿触发）

通信速度：普通为100kbps

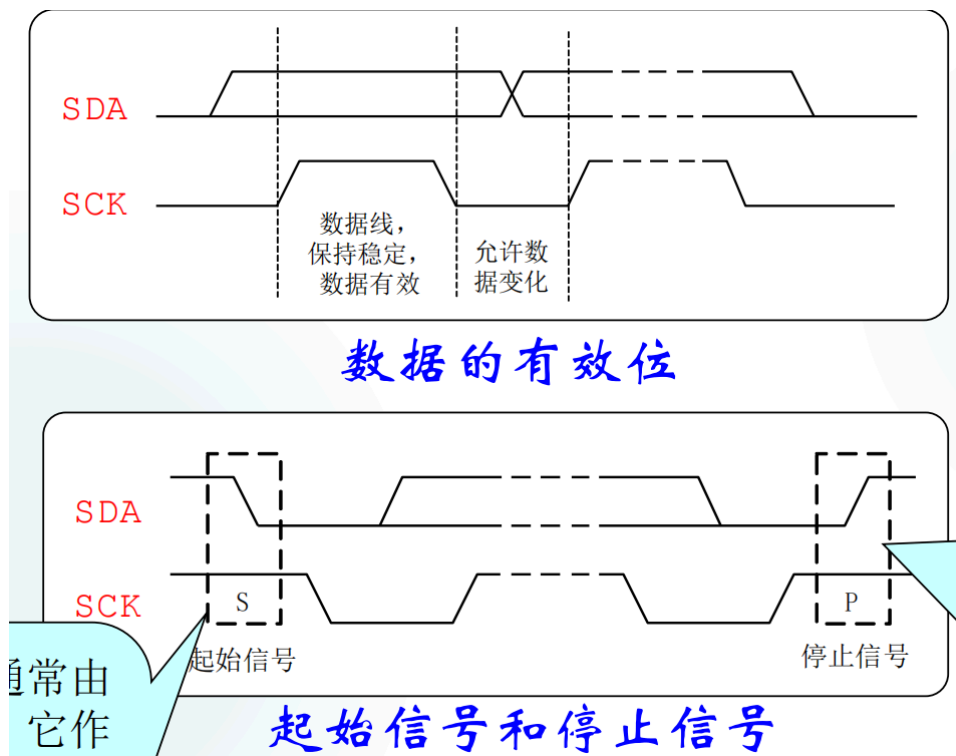
多主的主从式通信总线，2线制

- 外围器件：都有一个7位/10位的 从器件专用地址码
- 主控制器件：通过地址码建立多机通信的机制 不需要片选线
- 基于总线仲裁机制-----同一时刻只能有一个主端

SDA和SCL 为OD门 漏极开路-----必须接上拉电阻才能正常工作

总线信号：

- 起始信号：通常由主机发出 作为一次传输的开始。起始信号后总线 被认为是处于忙状态
- 停止信号：一次传送的结束 在该信号之后 总线被认为再次处于空闲状态



基本通信过程：

- 主机：发S
- 主机：发1字节的从机地址信息，最低位为读写控制码（1读 0写），高7位为从机器件地址代码
- 从机：发应答信号
- 主机：发数据，每发一字节---从机发应答信号给主机
- 主机：发P

带有7位地址的数据传送

- 先发送从机地址，其中第8位：数据传送方向
- 每个字节后必须跟一个应答
- 数据传送
 - 首先传输的是数据的最高有效位
 - 从机无法立即发送或接收下一个字节的时候，它可以将SCL线拉成低电平，从而使主机处于等待状态
 - 数据传输总是由主机生成一个停止条件而终止，主机可以在没有产生停止信号的时候，通过再产生一个开始信号---传输总线上另一个设备地址，而与之通信

带有10位地址的数据传送

- 10位从机地址是由起始条件或重复起始条件后的头两个字节组成

I2C每传送一个字节数据后必须有应答信号。主控器接收数据时，如果要结束通信-----在停止位之前发送非应答信号

SPI: 串行外设接口

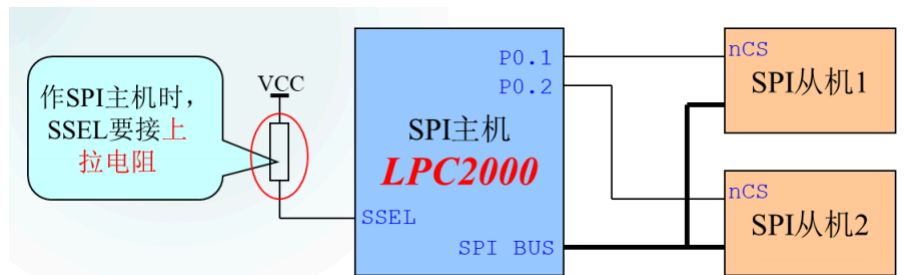
处理器与外设间的简单串行通信接口

- 全双工同步串行总线、一主多从式通信
- 两条数据线用于收发数据，时钟线用于同步
- 串行时钟由主机产生，当主机发送一字节数据（通过MOSI）的同时，从机返回一字节数据（通过MISO）

原理：通过移位或者循环移位

四个引脚

- SCK: 输入输出。串行时钟。用于同步，由主机输出
- SSEL (CE CS NSS) : 输入。从机选择。低有效，每个从机有不同的信号
- MISO: 输入输出。主入从出，MISO信号是一个单向的信号，从---主
- MOSI: 输入输出。主出从入，MISO信号是一个单向的信号，主---从



SPI控制寄存器SPCR1

- 只能在复位 初始化SPI时写一次
- SPIE: 中断允许位, 1使能 0禁止
- SPE: SPI使能位, 1使能 0禁止
- SPTIE: SPI传输中断允许位, 1 允许SPTE中断 0 禁止
- MSTR: SPI主从模式设置位, 1 主 0从
- **CPOL: 时钟极性位 0 正脉冲 1负脉冲**
- **CPHA: 时钟相位, 0-----第一个跳沿采样, 第二个跳沿输出**
- SSOE: 选择从设备的输出使能位
- LSBFE: 1为低位优先 0为高位优先

SPI的时钟极性和相位的配置----SPI模式

- 时钟极性：CPOL为0 时钟空闲的时候为低电平
- 时钟相位：CPHA为0 第一个跳变沿采样

UART：通用异步通信收发器

- 异步通信，全双工
- 两个器件不需要共享同一个时钟信号
- 单位：帧
- 工作模式：中断 DMA

UART：收发双方 相同的数据格式和波特率（传输速率）

UART0的相关寄存器

UART规范定义了：

- TTL电平的可配置通信数据格式
- 通信速率
- 异步采样机制

差分传输：两根线上都传输同一信号，二者振幅相同，相位相反（平衡传输方式）

接收端通过比较这两个电压的差值来判断发的是1还是0

差分走线要求：等长 等宽 紧密靠近 同一层面

典型工业总线：

- CAN总线：控制区域网络

七、嵌入式软件

计算机启动

- 通用计算机：

- 系统上电或复位，cs: ip的初值为一条跳转指令，跳转到ROM中BIOS的入口地址-----复位向量地址
- 启动BIOS。BIOS上电自检，无错误的话将第一个启动设备的第一个扇区加载到系统RAM的某个地址，启动MBR中的引导程序，进入引导的第一阶段
- Loader引导程序运行，进入引导的第二阶段
- 引导程序调用的OS Boot Loader 把用户选择的OS内核加载到内存，并跳转到OS入口地址执行，此时计算机控制权给了OS

总结：跳转到BIOS入口----启动BIOS----启动引导程序----引导程序运行----把OS内核加载到内存

- 嵌入式系统的引导与启动
 - MCU
 - 上电复位，pc初值为0x0000;
 - 0000h 0001h 0002h，这三个单元存放了一条无条件跳转指令，直接跳转到主程序的入口地址
 - ARM
 - 复位之后从0x000开始执行

嵌入式系统的启动略有差异，相同点如下：

- 基本地 跳转到入口地址
- 执行基本的引导程序初始化资源
- 在内存中 加载应用并启动执行

基于裸机的嵌入式软件

- 直接部署在嵌入式硬件上的软件-----裸机器代码
- 特点：
 - 功能简单 代码量小
 - 无限循环结构
 - 编写最开始的初始化代码，设定中断向量表，配置时钟，波特率，IO，中断优先级
 - 结构简单 额外资源开销小

ROM Monitor 监控程序

- 是运行在嵌入式系统中的一段代码，可以管理系统资源与用户进行交互
- 嵌入式OS正常的时候，不需要监控，故障的时候可以作为恢复系统的接口和手段
- 有的监控程序驻留在ROM中，提供给用户开发使用，如程序下载 调试 固化
- 目前嵌入式OS中的监控程序的作用已经减少

其基本功能

- 基本的系统初始化能力
- 简单的用户命令接口
- 支持程序加载和执行
- 设置断点 单步跟踪
- 可以查看reg和memory的值
- 可以与调试器集成
- 与硬件跳线配合

启动代码-----引导+初始化代码

- 系统加电后进入复位向量，执行启动代码
- 启动代码包括：引导代码 and OS执行环境的初始化代码

Boot loader 技术

- PC中，引导加载程序由BIOS和位于MBR中的OS Boot Loader组成
BIOS完成硬件检测和资源分配----把MBR中的Boot Loader读入内存----Loader把内核映像读入内存---内核入口点执行
- 嵌入式无固化的BIOS，主要由Boot Loader完成
初始化硬件、建立内存映射图、设置环境
- Boot Loader 与硬件密切相关，不完全通用

Boot Loader 基本说明

- Boot Loader 是驻留在嵌入式处理器的片上ROM或板上Flash的一段可启动代码
- ROM EEPROM FLASH 等被映射到0地址，Boot Loader 存在0地址，系统加电后，CPU首先执行 Boot Loader
- 具有最小功能要求：至少驱动一个外部的数据接口，至少一种读写 擦除 Flash的方法
- 不包括应用系统功能，可以和外界 通信，清楚系统内存的映射模型

Boot Loader的运行过程

- 单阶段运行
 - 在对Boot Loader 寻址或搬到SRAM中之后，Boot Loader完成全部任务，并引导运行
 - 因为SRAM容量有限----单阶段的Boot Loader 代码量小-----功能简单
- 多阶段运行
 - 第一阶段代码，为下一段启动代码 准备 加载 启动
 - 第二阶段代码，为下一段代码 准备 加载 以此类推 直到被启动
 - stage1:

- 汇编语言
- 简单的硬件初始化
- stage2:
 - C语言
 - 复杂功能
 - 设置启动参数
 - 串口通信等功能
 - 可读性和可移植性高
- stage1:
 - 硬件初始化----为加载boot Loader的stage2准备ram空间----拷贝boot Loader的stage2到ram空间----设置好堆栈----跳转到stage2的入口点
- stage2:
 - 硬件初始化----检测系统内存映射----将内核映像和根文件系统映像从Flash加载到ram----为内核设置启动参数----调用内核

□ stage1 通常包括以下步骤

- 1硬件设备初始化
- 2为加载 Boot Loader 的 stage2 准备 RAM 空间
- 3拷贝 Boot Loader 的 stage2 到 RAM 空间中
- 4设置好堆栈
- 5跳转到 stage2 的 C 入口点

□ stage2 通常包括以下步骤

- 6初始化本阶段要使用到的硬件设备
- 7检测系统内存映射(memory map)
- 8将kernel映像和根文件系统映像从flash加载到RAM中
- 9为内核设置启动参数
- 10调用内核

1.硬件初始化：屏蔽所有的中断，设置CPU的速度和时钟频率，关闭CPU内部指令/数据cache

2.准备ram空间：要确保所安排的地址范围是 可读写的 ram空间

3.拷贝stage2到ram：确定可执行映像在全局存储设备的存放起始地址和终止地址；确定ram空间的起始地址

4.设置堆栈指针sp：通常把sp=stage2_end +4

5.跳转到stage2的入口

6.初始化本阶段要使用的硬件设备：初始化至少一个串口来IO输出。初始化计时器。变量使用volatile，避免编译器优化

7.检测内存映射：嵌入式系统把CPU预留的全部RAM地址空间中的一部分映射到RAM单元，其余unused。stage2**必须**检测整个系统的内存映射情况，来知道哪些被映射 哪些是unused

8.加载内核映像和根文件系统映像：规划一下内存占用的布局，从Flash上拷贝内核映像到ram上

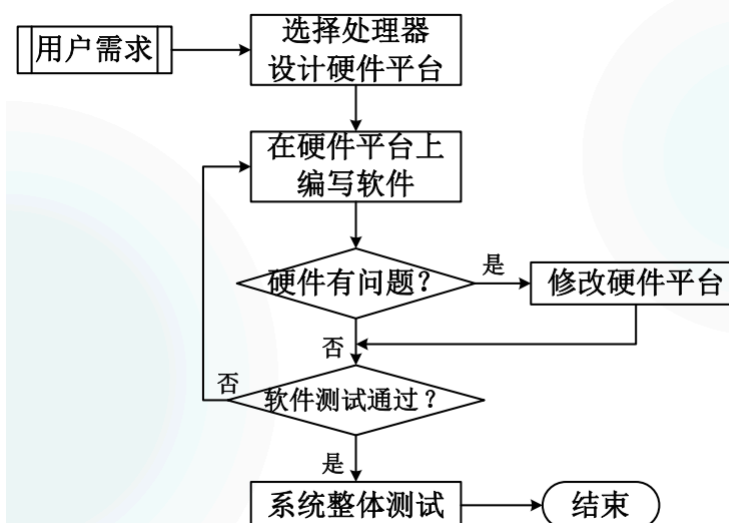
9.设置内核的启动参数

10.调用内核：直接跳转到内核的第一条指令处，跳转的时候要满足的条件：

- CPU寄存器I的设置
 - CPU模式：禁止中断，管理模式
 - Cache和MMU的设置：MMU和数据cache关闭，指令cache
-
- BSP
-
- 虚拟机监控程序：管理硬件资源并向上虚拟出新的资源，并提供管理服务
 - 常见类型：OS就是一类虚拟机；独立的虚拟机系统

嵌入式软件的基本设计方法

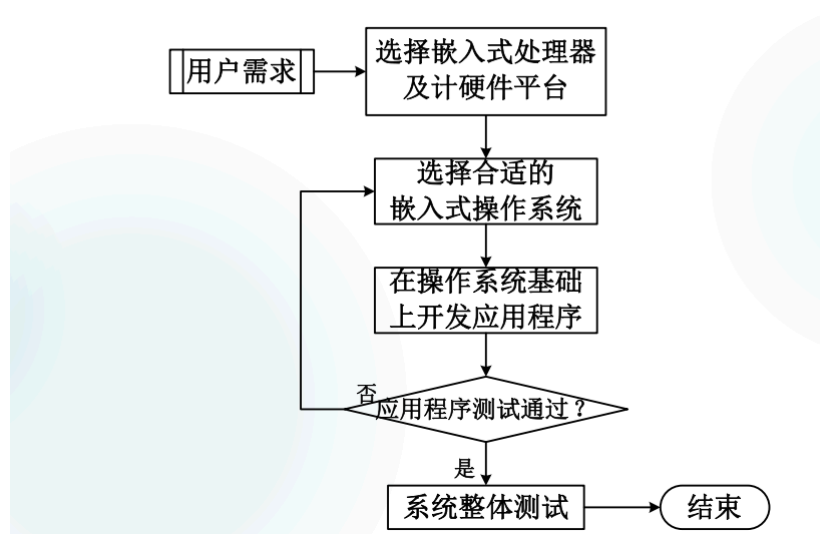
- VS通用计算机：软件开发中的开发平台和运行平台运营。完成开发就可以运行
- 嵌入式：大部分的开发平台和运行平台不同：
 - 需要模拟环境，交叉编译环境
 - 开发的software需要经过处理 ----才能在目标平台运行
- 基于裸机的软件开发：
 - 无OS，系统架构简单
 - 多中断来实现简单的多任务运行
 - 软件与特定的硬件结合-----难以在其他平台运行



嵌入式操作系统：EOS -----嵌入式硬件和软件的中间层

- 对系统软硬件资源管理
- 支持多任务调度
- 向上提供API -----软件不要管理底层的硬件，开发与调试方便
 - eg：多任务软件，用户只需要创建多任务，分配优先级，设置通信方式
 - eg：临界区管理，用户采用EOS中的信号量
- EOS的复杂程度取决于-----应用需求----是否实时 是否抢先式调度 是否引起优先级翻转

复杂计算系统开发流程

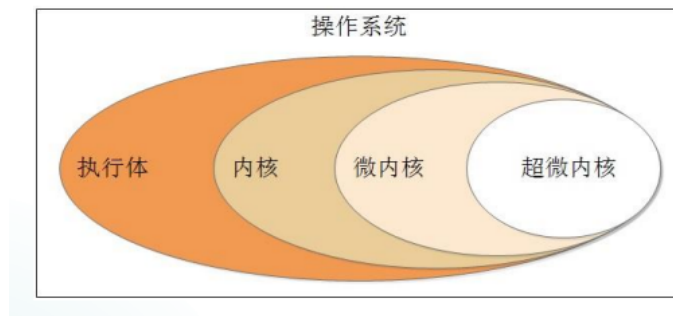


八、嵌入式OS

- 内核
 - OS的核心组件，一个可以独立运行的软件对象，与OS的其他组件协同工作来提供系统级服务
 - 提供： 任务管理 同步 调度及定时器 中断与异常处理 基本的内存管理
- 微内核and超微内核架构
 - 将共性核心功能与可选组件进行分离
 - 内核最早被加载，提供任务调度 任务间通信 中断/事件分发 等基本的系统服务 并以其为中心 向外扩展软件功能
 - 优点：
 - 结构灵活 关键代码分区存放 防止非法访问
 - 比单内核具有核心更小 组件化程度高 使用更灵活

EOS采用微内核/超微内核

- 单核处理器上的超微内核是一个 事件分发器 -----最先捕获系统的中断和事件，进而将其分发给上层的系统内核或应用
- 多核处理器：超微内核要根据 需要 来 分配 处理器的计算核资源



超微内核：中断 事件分发 多核处理器资源分配

微内核：+任务调度

内核：+同步

执行体：包括了 私有内存块 IO服务

OS：提供了用户交互接口 文件系统 数据库 安全服务 -----是一个完整的系统软件

实时性与实时系统

- 实时：系统能够在限定的响应时间内提供所需水平的服务
- 实时系统：计算的正确性不仅取决于程序的逻辑正确性，也取决于结果产生的时间，如果系统的时间约束条件得不到满足----系统出错
- 要求：逻辑正确 and 时间正确

可调度：每一个具有截止期要去的任务，只要在释放时间后启动，就能够在截止期之前ok

可调度性：对于一个调度算法，一组任务总是有可行的调度方案。如果一个系统中的所有任务都是可调度的-----系统是实时的

错失率：已执行任务中，完成执行但超过截止期的任务占比

丢失率：丢弃的任务比例

失效率：错失率+丢失率

硬实时：

- 截止期必须被满足
- 硬实时系统：飞行控制，核电站控制

软实时：

- 软截止期允许被错过
- 软实时系统：多媒体应用 视频监控 城市交通调度

内核服务机制：

- **任务管理**
 - 是OS内核的核心，包括任务创建 调度 删除
 - 任务的基本操作
 - 任务创建：在内存中为任务控制块和栈分配空间，进而初始化并激活任务控制块
 - 任务删除：结束执行，删除任务实体 释放占用资源 或 挂起任务
 - 任务控制：软件中通过系统调用，对任务运行状态 进行动态控制
 - 任务：一个独立功能的无限循环的程序段的一次运行活动----实时内核调度的单位
 - 动态性：任务状态是不断变化的：就绪态 运行态 等待态
 - 并行性：系统同时存在多个任务，宏观是并行的
 - 异步独立性：每个任务各自按相互独立的不可预知的速度运行
 - 任务的内容：代码 数据 堆栈 上下文环境 无限循环的程序
 - 上下文环境：任务优先级 任务状态 寄存器的值
 - 上下文环境过 任务控制块 TCB 来体现
 - 任务的分类：
 - 达到情况的可预测性：周期任务 非周期任务
 - 重要程序：关键任务 非关键任务
 - 任务参数：
 - 任务的优先级-----分为静态优先级（优先级在过程中不改变）和动态优先级
 - 周期
 - 任务的计算时间：通常用最坏情况下的执行时间，也可用统计时间
 - 任务的就绪时间：任务具备了在处理器上被执行所需要条件的时间
 - 任务的截止时间：绝对截止时间 and 相对截止时间
 - 强截止时间（关键任务 强实时）
 - 弱截止时间（弱实时）
 - 任务的状态：等待 就绪 运行
 - 空闲任务：无其他任务时执行空闲任务，其为最低优先级，软件无法删除空闲任务，其由OS创建
 - 任务控制块：TCB，包含任务相关信息的数据结构，如名字 起始地址 优先级 状态 硬件上下文 队列指针

- 为了节约内存-----任务数量要预先配置。一个任务对应一个TCB，形成一个空闲任务控制块链
 - 创建一个任务，实时内核从空闲任务状态链中为任务分配一个TCB。删除的话，回收TCB到链中
 - 任务切换：保存当前上下文，并恢复需要执行的任务的上下文过程
 - 当前的上下文 放到当前任务的TCB
 - 把需要运行的任务的上下文从对应的TCB恢复出来
 - 基本步骤：
 - 保存任务上下文环境
 - 更新当前运行任务的TCB内容-----状态为就绪或等待
 - 把TCB放入就绪队列或者等待队列
 - 选择另一个任务执行
 - 改变新任务的TCB-----运行状态
 - 恢复新任务的上下文环境
 - 时机：
 - 中断 自陷
 - 自陷：由执行任务中当前指令所引起，导致去处理错误或异常
 - 缺资源 阻塞
 - 时间片轮转调度
 - 高优先级任务 就绪：基于优先级的 抢占式调度
 - 时间：保存时间+调度时间+恢复时间
 - 任务队列：在队列中如何获得最高优先级的就绪任务-----1.每次遍历-----2.进入的时候从高到低排序
 - 上述不确定性，提出-----优先级位图（[优先级位图算法详解-CSDN博客/uCos中的任务机制和优先级位图算法-知乎\(zhuhu.com\)\)](#)）
-
- 任务调度
 - 典型
 - 单调速率调度RM
 - 单调截止期调度DM
 - 最早截止期调度EDF
 - 轮转调度RR
 - 调度程序是一个函数调用 而不是一个任务
 - 实时调度的划分
 - 离线 在线
 - 抢占 非抢占
 - 静态 动态

- 最佳 试探性
- 抢占式调度（实时内核大多采用，为了使关键任务能够打断非关键任务，确保关键任务的截止时间满足）and非抢占式调度
 - 抢占式调度---低优先级长时间得不到执行
 - 非抢占式调度---用于任务需要按照预先确定的顺序执行的情况
- 内核的可抢占性：执行内核提供的系统调用时，是否可以被中断打断
 - 可抢占内核：比如正在执行内核服务函数，也能响应中断，并且中断服务程序ISR退出时可以任务重新调度
 - 不可抢占式内核的两种情况
 - 内核服务函数不能被中断（即处于关中断状态）
 - 能被中断 但是不能进行任务重新调度
- 基于优先级的可抢占式调度
- 时间片轮转调度
 - 时间片太大---没有意义
 - 时间片太小---切换频繁，处理器开销大，任务本身运行时间减小
- 优先级翻转
 - background：低优先级任务持有高优先级任务所需的临界资源----高优先级任务被延迟执行----优先级翻转问题
 - example：1和3用一个信号量 1---3---3要资源---1---但是1可能被中优先级2抢占---使得3等待的时间更长
 - solution：两个协议
- 优先级继承协议
 - 问题发生的时候-----让持有共享资源的低优先级任务获得被阻塞高优先级任务的优先级
 - 具体
 - A阻塞了BCD，A的优先级为MAX（B,C,D）
 - A退出临界区的时候，A的优先级为A
 - 注意不是等A全都执行完毕，而是等A不用临界资源的时候就over
 - 系统在运行前就可以确定任务的 最大阻塞时间
 - 问题：
 - 不能避免死锁
 - example：1拿到了S1-----1还想要拿S2，但是2来了并把S2拿走了-----2想要S1-----但是拿着S1的1被阻塞---死锁
 - 任务阻塞时间有界，但可能出现阻塞链----使得任务的阻塞时间会更长
 - example：1拿到了S1-----2来了抢占调度，并拿到S2-----3需要顺序的S1和S2----但是S1和S2分别被占---阻塞链
- 优先级天花板协议
 - 解决死锁和阻塞链问题
 - 信号量的天花板为能申请到该信号量的所有任务中具有最高优先级任务的优先级
 - 如果任务获得信号量，则任务的优先级为该信号量的优先级天花板。执行完临界区之后--恢复优先级

- 两个协议的对比：
 - 优先级继承协议可能多次改变占有某S的Task的优先级，而优先级天花板协议只需要改变一次
 - 所以后者的效率更高----因为更小的额外开销-----任务执行临界区的时间减少
 - 前者对执行流程的影响较小；后者可能影响某些中优先级任务的完成时间
- 任务间通信
 - 通信方式
 - 共享内存区域：多个任务写操作---互斥
 - 消息队列：内核中为多任务交互提供的以消息为单元的数据交互机制
 - 可以有多个消息队列
 - 每个队列有id，队列中的消息数量和长度可变
 - 支持FIFO队列和优先级消息队列
 - 消息邮箱：相比队列，消耗资源少，运行速度快
 - 管道：
 - 在任务之间提供 消息流 的IO设备
 - 允许写入的消息长度不受限制。而消息队列和邮箱都要求写入固定长度消息
 - 完全面向字节流，可以按M个字节写入，按N个字节读出，每次读写的字节长度可以不同
 - 管道比队列和邮箱多的一个重要特性：可以使用标准的select函数来读管道
- 任务间协同----同步
 - 事件的定义
 - 信号量的操作方法：
 - 使用临界资源之前，得到信号量
 - 使用临界资源
 - 使用完之后，释放信号量
 - 信号量
 - 二进制信号量：full empty
 - 互斥信号量
 - 计数信号量：记录信号量的释放次数，释放一次++，获取一次信号量---。计数器为0代表想要获取信号量的任务将被阻塞
 - 任务信号量
 - 信号：即 软中断信号 异步
 - 对比
 - 信号量最快 最简洁 但只能传递一位信息
 - 事件比信号量复杂，更多的处理器时间，优势：一个任务可以等待在多个事件上

- 队列可以传递大量信息，比事件要灵活。缺点：读写消耗更多的处理器时间。邮箱和管理也是一样
- 中断与异常处理
 - 异步机制
 - 中断发生后-----处理器识别中断请求----跳转到盖中断请求对应的中断服务程序执行
 - 中断检测 中断响应 中断处理 的过程可以 全部由处理器独立完成
 - 中断管理
 - 硬件中断：异步事件，是处理器机制
 - 软件中断，异常：是同步事件，是OS内核中扩展、实现的机制
- 存储管理
 - 静态内存管理
 - 提前将内存空间分段划分为多个区域
 - 三大区域：代码区 数据区 可动态使用的存储区域
 - 动态内存管理
 - malloc and free
 - 优点：灵活性强 内存利用率高
 - 缺点：内存碎片 分配失败

十、嵌入式软件设计与系统分析

设计流程

- 自顶向下
- 自底向上

开发流程

- 需求
- 规格说明
- 体系结构
- 组件

- 系统集成
- 系统测试

一些题:

某系统通过UART接口进行7位ASCII码字符传送，含有1位偶校验位和2位停止位，当波特率为115200时，字符传送率（即每秒传输的字符数）为_____。

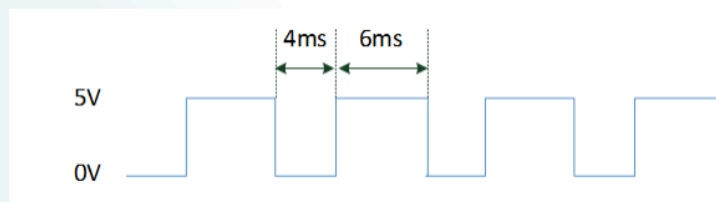
A、115200 B、14400 C、16457 D、11520

一个字符是 $7+1+2=10$ 位

波特率115200-----每秒115200位

所以每秒11520个字符

下图为某PWM信号，请问该PWM占空比是多少？简要解释PWM原理。



60%

PWM是脉冲宽度调制。-----对一系列脉冲的宽度进行调制-----来等效获得所需要的波形和数值

PWM信号是一些列高低电平交替的脉冲组成，通过改变高电平持续时间（脉冲宽度）与占空比，可以控制输出信号的平均电压等

如电机调速 呼吸灯 控制舵机

请举出一个嵌入式计算系统的例子，并说明它不是通用计算系统的原因。

智能手环

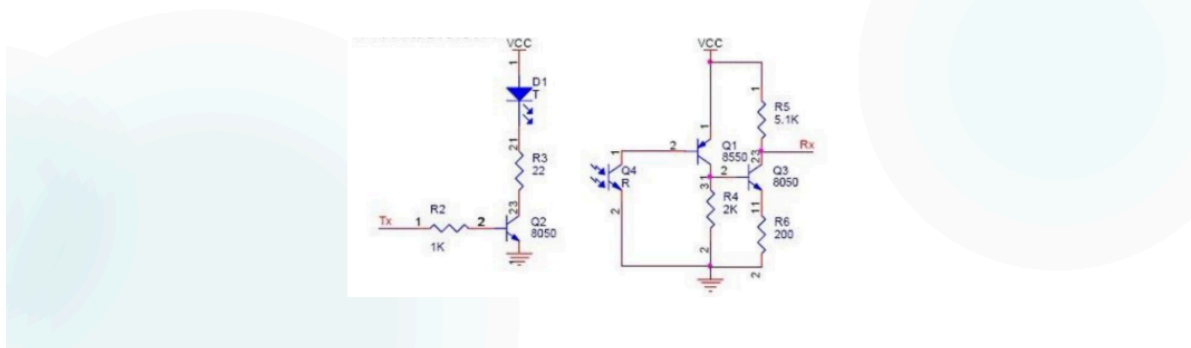
功能专一--健康监测

资源受限

定制化软件-----针对功能和硬件定制

红外对管的原理图类似下图（左侧是红外发射电路、右侧是红外接收电路），其中Tx、Rx均连接至GPIO口。试分析：

- （1）Tx如何控制发射管工作？
- （2）当发射管工作时，Rx的电平如何变化？



TX高电平-----三极管的基极导通，自上而下电流，D1有电流

TX低电平-----三极管的基极不导通，截止，D1无电流通过

发射出的红外光被Q4接受 光电效应--导通，Q3也会导通，原本上拉至vcc的rx引脚电位被拉低，Rx低电平

当发射管不工作的时候，Q4截止，Q3截止，Rx引脚通过电阻R5上拉到vcc，所以高电平

某公司要开发一个便携式人脸识别设备用于室内安检，请帮助设计一个满足需求的嵌入式系统，以实现快速、准确的人脸识别功能。其中，主要部件可从下面列表任选，其他必要部件自行设计。

处理器：（1）高性能PC机（2）高性能嵌入式处理器，如ARM Cortex-A系列处理器（3）DSP处理器（4）高性能SoC，如英伟达jetson Nx

存储器：（1）EEPROM（2）硬盘（3）Nand Flash（4）DRAM

操作系统：（1）ubuntu（2）鸿蒙LiteOS（3）windows（4）VxWorks

请给出系统设计方案，并说明这么设计的理由。

- **处理器**：选择英伟达 Jetson Nx 高性能 SoC

专门针对边缘计算和嵌入式视觉应用设计的，集成了GPU，适合处理图像数据，加速人脸识别算法的CNN计算

相比ARM 图形和并行计算能力更强

DSP主要针对数字信号处理

高性能PC机 体积大 功耗大

- **存储器：**搭配 Nand Flash 和 DRAM 。Nand Flash 用于存储程序代码、人脸识别模型等数据；
DRAM 作为运行内存，为处理器提供快速的数据读写空间，支持系统和程序运行

nand flash：非易失性 存储容量大 成本较低 适合存储大量程序代码，模型参数等静态数据

dram：高速读写 快速响应

而eeprom 容量小

硬盘 体积大 功耗高

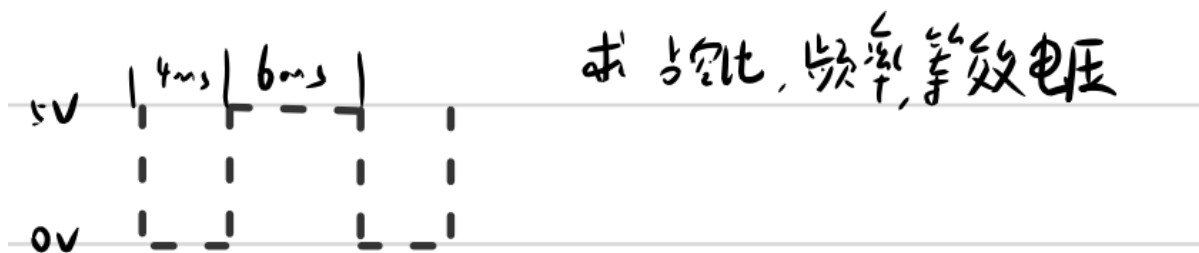
- **操作系统：**选用 Ubuntu

有大量成熟的人脸识别开源库 和 深度学习框架 社区活跃

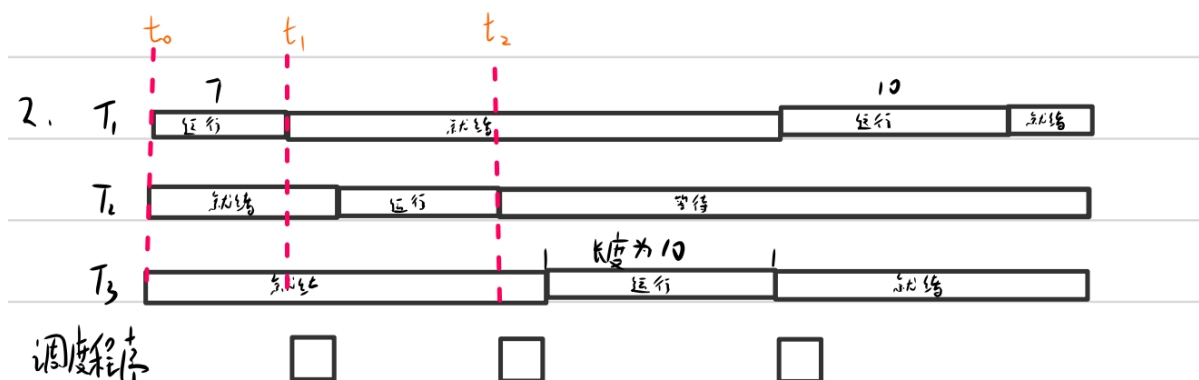
鸿蒙 侧重于轻量级物联网设备

windows 资源占用大 定制型相对较差

vxworks 是实时系统 但是软件生态丰富度不如Ubuntu



占空比60%，周期10ms---频率为100hz，等效电压为 $0.6 \times 5 = 3$



t_0, t_1, t_2 发生了什么, 发生该现象可能的原因

t_0 : T1运行 T2 T3就绪。可能是优先级调度 则T1最高，也可能是时间片轮转 正好轮到T1

t1: T1变就绪 T2变运行。如果是时间片轮转, T1的时间片用完。

t2: T2等待 T3运行 T2需要资源 去让T3执行

4. 当ARM处理器执行到指令地址为X的指令时, 收到I/O请求, IRQ中断异常处理后, 返回的地址是? IRQ是如何实现返-返回? 这样实现的原因?

响应中断时, 处理器暂停当前指令, 保存当前处理器状态和下一条指令地址X+4, 切换到IRQ模式并跳转至中断处理程序执行相关操作。处理完成后, 处理器恢复保存的处理器状态, 并依据保存的下一条指令地址, 回到地址为X的指令的下一条指令处继续执行, 以此保证程序执行的连续性, 维持处理器状态一致性, 确保主程序稳定正确运行。

6. 已知 $OSRdyGrp = 0x \dots$, $OSRdyTbl[1] = 0x \dots$, $OSRdyTbl[4] = 0x \dots$, $OSRdyTbl[6] = 0x \dots$
给出 $OSVnMapTbl$ 的大表, 问当前最高优先级任务, 优先级位图的意义

见上面的链接

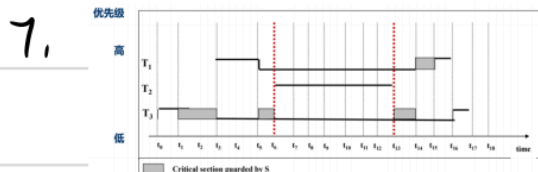


图1

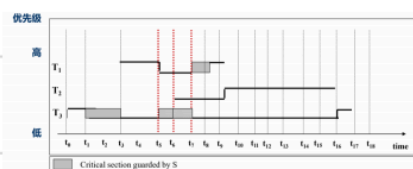


图2 优先级继承

1) 通过图1说明什么是优先级反转

12) 图2 t_5 发生了什么 (包含任务切换), 继承的优先级?

t_3, t_1 发生了什么 (包含任务切换)

13) 画出优先级天花板的时序, 说明 t_3, t_5 发生了什么

设计一个交通灯有限状态机，红灯黄灯停，绿灯行，红灯在输入tick有效变为green，输出go，在黄灯tick变为yellow，输出stop，再次触发tick，变为red，输出stop

```
void handleTick() {
    switch (currentState) {
        case RED:
            currentState = GREEN;
            printf("go\n");
            break;
        case GREEN:
            currentState = YELLOW;
            printf("stop\n");
            break;
        case YELLOW:
            currentState = RED;
            printf("stop\n");
            break;
        default:
            break;
    }
}
```

1. 2010 ~ 现在的计算模式, (多选) _____

A 物联网, B 云计算 C 雾计算 D 边缘计算

abcd

ARM 取 0x20000 处数据 0x55220000

地址	20000			
数据	55	22	00	00

正确的是 ()

A 大端存储 B 小端存储 C ARM 状态 D Thumb 状态

小端是高位放在高地址，低位放在低地址，所以A

其中arm的指令字长为32位，四字节；而thumb为16位 两字节

5, 下面不属于执行器

A. 伺服电机

B ...

C 继电器

D 电子标签

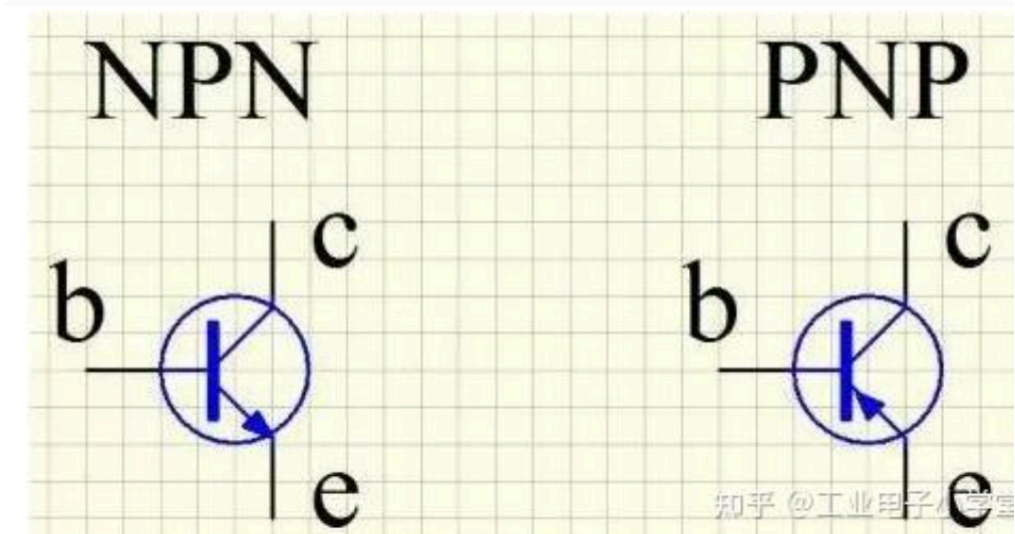
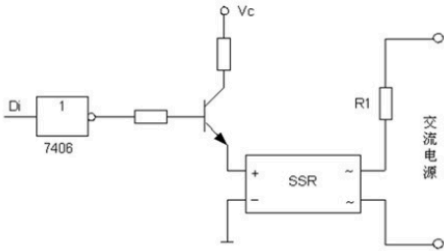
d

下图中三极管属于什么类型？

- A

PNP型
- B

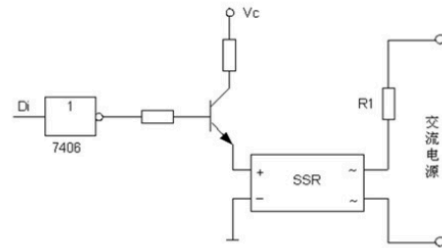
NPN型



发现为NPN型 所以为B

图中Di为 () 时, SSR导通

- ☐ A 高电平
- ☐ C 低电平



di为低电平-----反向之后为高电平-----三极管导通。所以选C

在嵌入式系统的存储器设计中, 以下哪项不是需要综合考虑的因素?

- ☐ A 存储器类型与容量规划
- ☐ B 存储器的读写性能
- ☐ C 存储器的可靠性和稳定性
- ☐ D 存储器的物理尺寸和颜色

不需要考虑D

在嵌入式系统的存储器中, 以下哪种存储器不需要刷新且功耗低?

- ☐ A DRAM
- ☐ B SRAM
- ☐ C SDRAM
- ☐ D DDR SDRAM

A---电容---刷新---功耗高

B---双稳态电路，只要供电---就保存数据 功耗低（选B）

C---动态存储器 需要同步信号来读写 电容 刷新 功耗高

D--- 在SDRAM 上提高数据传输速率